

Introduction

- Overview
- Getting Started
- Automatic Install
- Manual Install
  - Install Kubernetes
  - Run Kubernetes
  - Prepare Kubernetes
  - Deploy Entando
  - Login to Entando
  - Next Steps
- Solution Templates
- Entando Architecture

ENTANDO CLI

Create Components

Curate and Share Components

Compose an Application

Consume an Application

Reference

Community

Release Notes

Getting Started with Entando

WARNING

If you have a MacBook with an M1 or M2 chip, please see our alternative install guides. Entando 7 is not currently compatible with an ARM64-based architecture.

You can set up Entando in two simple steps or install it manually to meet your specific needs.

- Automatic Install:** The fastest way to locally install and start up Entando in Kubernetes.
- Manual Install:** Useful if you want a shared cluster or a custom local instance.

Automatic Install

Automatically install Entando via the Entando command-line interface (CLI) and prepare a local developer environment with default settings. The following steps launch an Ubuntu VM via Multipass, install Kubernetes, then deploy Entando to it.

1. Install [Multipass](#)
2. Install Entando into Kubernetes on Ubuntu using the [Entando CLI](#)

```
curl -sL "https://get.entando.org" | ENTANDO_RELEASE="v7.1.6" bash
```

3. The progress of the install is displayed on the console. Installation can take up to 10 minutes, depending on how long the Docker images take to download. The sequence of steps performed by the CLI is identical to the manual steps below. To understand what the CLI does, review the manual steps.
4. The URL to access the Entando App Builder will print to the console once the install completes.
5. Login with username: `admin` and password: `adminadmin`. Refer to [Login to Entando](#) for more information and next steps.

Manual Install

Manual installation allows you to configure a shared cluster or customize a local developer environment. The following learn-as-you-go approach provides a working knowledge of Kubernetes as you install Entando in a local environment. After downloading the necessary files, your instance of Kubernetes will be up and running in <60 seconds.

This is a three stage process:

1. [Install Kubernetes](#)
2. [Prepare the Kubernetes Environment](#)
3. [Deploy Entando](#)

WHAT'S NEEDED TO RUN KUBERNETES?

Kubernetes is a container orchestrator designed to manage a server cluster. It requires at least one master node running a Linux OS. A lightweight Ubuntu virtual machine (VM) can be created in seconds with Multipass. Choosing a Type 1 hypervisor eliminates a guest OS, maximizing speed and performance.

Install Kubernetes

Enable Hypervisor

WHY A HYPERVISOR?

Hypervisors allow you to create and run VMs. Virtualization software that run on top of your operating system as "guests" are Type 2 hypervisors, e.g. VirtualBox or VMware Workstation. Type 1 hypervisors run directly on your host machine.

Install a Type 1 hypervisor for optimal performance.

Mac: Install [hyperkit](#)

```
brew install hyperkit
```

Windows: [Install Hyper-V](#)

► What if my machine doesn't support hyperkit or Hyper-V?

Launch an Ubuntu VM

WHY MULTIPASS?

Multipass is a cross-platform tool developed by the publishers of Ubuntu to create lightweight Ubuntu VMs in seconds.

1. Install [Multipass](#)

2. Launch a VM

```
multipass launch --name quickstart --cpus 4 --mem 8G --disk 20G
```

3. Open a VM shell. (Note: on Windows, you may need to use `winterpty multipass` )

```
multipass shell quickstart
```

Run Kubernetes

WHY K3S?

K3s is a certified Kubernetes distribution designed for production workloads in resource-constrained environments.

It's packaged as a single <50MB binary that minimizes the dependencies and procedure required to install, run and auto-update a production Kubernetes cluster.

1. Install the version of `K3s` supported by Entando 7.1. Note that this may not be the latest version of Kubernetes.

```
curl -sfl https://get.k3s.io | INSTALL_K3S_CHANNEL="v1.23.14+k3s1" sh -
```

2. Check that the cluster `STATUS` is `Ready` .

```
sudo kubectl get node
```

► What's running out of the box?

CONGRATULATIONS!

You now have a local instance of Kubernetes up and running.

Now that Kubernetes is running, you can use `kubectl` to send commands directly to K3s from the host machine, rather than from within the VM. To set this up with the `ent CLI`, run `ent attach-vm quickstart` and then use `ent kubectl` for any calls to K8s. Alternatively, see the K3s documentation to [access your cluster with kubectl](#) .

Prepare Kubernetes

To install Entando, we'll add `Custom Resources` , create a `Namespace` and configure external access to our cluster.

Create a Namespace

WHAT ARE NAMESPACES?

Kubernetes supports multiple virtual clusters backed by the same physical cluster. These virtual clusters are called [namespaces](#) . You can use namespaces to allocate resources and set CPU/memory limits for individual projects or teams.

```
sudo kubectl create namespace entando
```

Add Custom Resources

WHY CUSTOM RESOURCES?

Standard resources in Kubernetes include `Pods` , which are groups of one or more containers, `Services` to call or access Pods, and `Ingresses` to enable external access to Services.

[Custom resources](#) let you store and retrieve structured data. Combining a custom resource with a custom controller allows you to define a desired state to automate how your applications or services run in a Kubernetes cluster.

Examples of custom resources in Entando are the [Entando App Engine](#) and the [Entando Identity Management System](#).

From your Ubuntu shell

1. Install the cluster-scoped custom resource definitions (CRDs)

```
sudo kubectl apply -f https://raw.githubusercontent.com/entando/entando--releases/v7.1.6
```

2. Install the namespace-scoped custom resources

```
sudo kubectl apply -n entando -f https://raw.githubusercontent.com/entando/entando--rele
```

Configure Access to Your Cluster

WHAT ABOUT NETWORKING?

Entando sets up `Ingresses` in Kubernetes to expose HTTP routes from outside the cluster to services within the cluster. This is used to access Entando from a local browser.

If you run into network issues during startup, or if you are using Windows for your local development instance, you'll need to [troubleshoot your network](#). Indications of network issues can include Entando failing to completely start or a working Entando instance failing to restart later.

To set up external access to your cluster, replace YOUR-HOST-NAME with the fully qualified domain of your Ubuntu VM.

- On Windows with Hyper-V, YOUR-HOST-NAME is the name of your VM followed by `.mshome.net` , e.g. `quickstart.mshome.net` .
- On Mac, Linux, or Windows without Hyper-V, you'll use a host name based on your VM's IP address. You can determine the IP address from within the VM with this command:

```
hostname -I | awk '{print $1}'
```

- Your IP-based YOUR-HOST-NAME should follow this pattern: `quickstart.YOUR-IP.nip.io` , e.g. `quickstart.192.168.64.33.nip.io` . The suffix `.nip.io` makes use of the free [nip.io](#) DNS service so that any requests to this host name will resolve to your VM. The prefix `quickstart` is arbitrary so you can choose your own.

Now that you've determined YOUR-HOST-NAME, use it to configure the Entando Application.

1. Download the template `entando-app.yaml` .

```
curl -sLO "https://raw.githubusercontent.com/entando/entando--releases/v7.1.6/dist/ge-1-
```

2. Modify `entando-app.yaml` to set the `ingressHostName` to YOUR-HOST-NAME. Examples:

- `ingressHostName` : `quickstart.mshome.net`
- `ingressHostName` : `quickstart.192.168.63.33.nip.io`

EMBEDDED DATABASES

To speed up the [Getting Started](#) environment, embedded databases are used by default. See this [Tutorial on Default Databases](#) for information on how to change your database connection, or you can modify the `dbms` setting in the `entando-app.yaml` .

ENTANDO OPERATOR

An optional ConfigMap can be used to modify the behavior of the Entando Operator. For example, on a slower network, you may want to increase the download timeouts. Refer to the [Entando Operator](#) page for more information.

ENTANDOAPP INGRESS PATH

To customize the ingress path of a deployment, refer to the [Configure the Entando App Context](#) tutorial.

Deploy Entando

Deploy Entando by applying `entando-app.yaml` to your namespace.

```
sudo kubectl apply -f entando-app.yaml -n entando
```

Use the `get pods --watch` command to observe Entando starting up.

```
sudo kubectl get pods -n entando --watch
```

► What does a successful startup look like?

Press `Ctrl-C` to exit the `watch` command once everything is up and running.

► What Pods come out of the box?

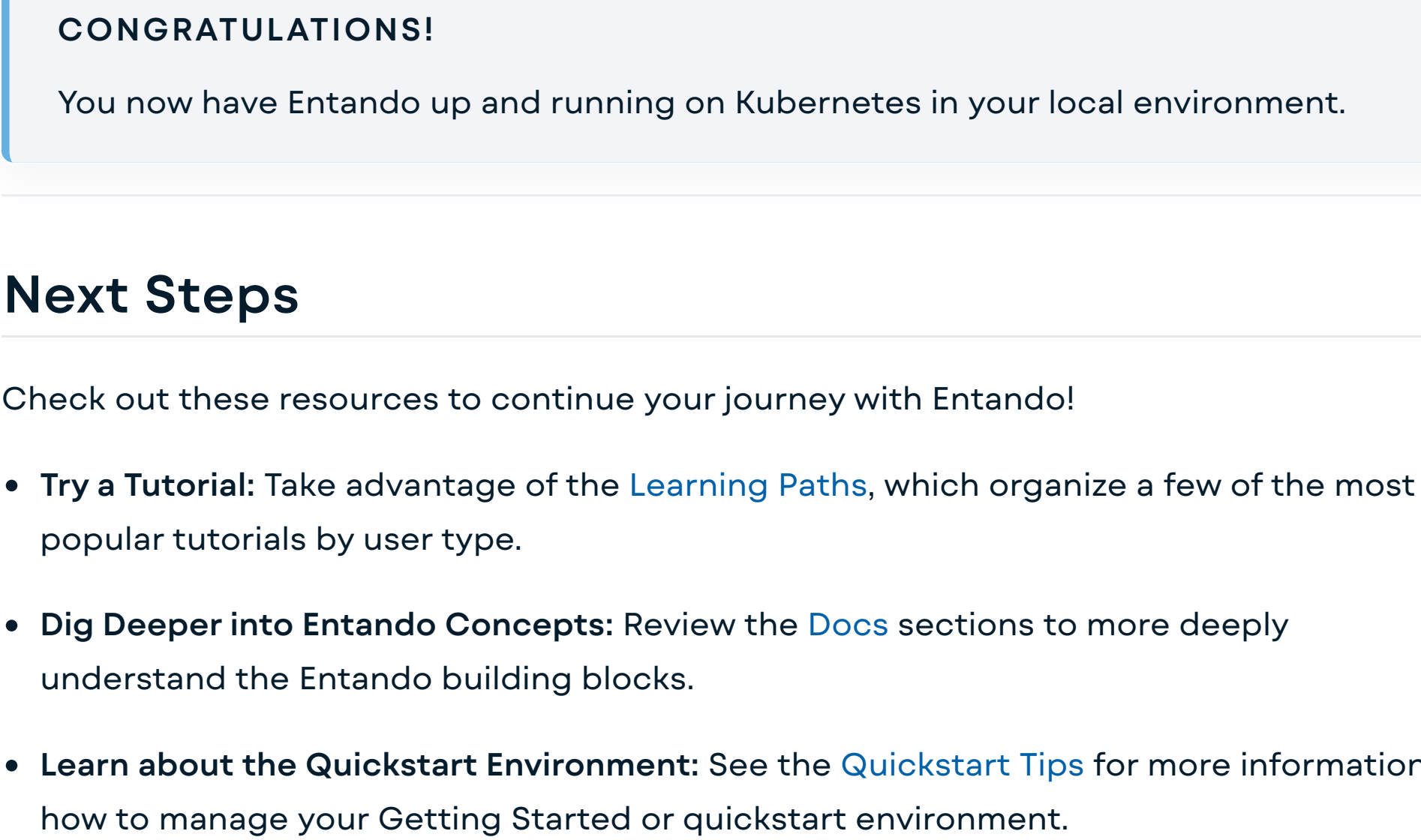
The URL of your running Entando instance is `http://YOUR-HOST-NAME/app-builder/`.

- Example URL

```
quickstart.192.168.64.33.nip.io/app-builder/
```

Login to Entando

Now that you've installed Entando, login to the Entando App Builder.



- Username: `admin`
- Password: `adminadmin`

After login, change your password to activate your account.

- Note: If the login process hangs for more than 5 seconds, refresh the browser.



In the App Builder, applications are composed from various components, including micro frontends and CMS content.

CONGRATULATIONS!

You now have Entando up and running on Kubernetes in your local environment.

Next Steps

Check out these resources to continue your journey with Entando!

- **Try a Tutorial:** Take advantage of the [Learning Paths](#), which organize a few of the most popular tutorials by user type.
- **Dig Deeper into Entando Concepts:** Review the [Docs](#) sections to more deeply understand the Entando building blocks.
- **Learn about the Quickstart Environment:** See the [Quickstart Tips](#) for more information on how to manage your Getting Started or quickstart environment.

Edit this page on GitHub

Last Updated: 2/14/2023, 11:34:34 PM